DATA SCIENCE / GUIA TÉCNICO EP

Dicionários e Funções

Contratos, parâmetros, escopo, qualidade e testes

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Dicionários e modelos de dados • 2. Funções com contrato explícito • 3. Parâmetros e argumentos mutáveis • 4. Escopo, pureza e testes • 5. Parâmetros e assinaturas • 6. Funções de ordem superior • 7. Erros, logs e contratos • 8. Testes unitários e parametrização

Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

01 Dicionários e modelos de dados

Dict associa chaves hashable a valores. get é apropriado para chaves opcionais; acesso por colchetes torna ausência obrigatória visível via KeyError. items percorre pares sem buscas repetidas.

Exemplo executável

```
aluno = {"nome": "Lia", "notas": [7.5, 8.0]}
media = sum(aluno["notas"]) / len(aluno["notas"])
assert aluno.get("turma") is None
for chave, valor in aluno.items():
    print(chave, valor)
```

Atenção técnica. None e chave ausente não são necessariamente equivalentes; use sentinelas quando a distinção for importante.

Exercício de consolidação. Crie contagem de palavras ignorando maiúsculas/minúsculas.

02 Funções com contrato explícito

Uma função deve ter entradas e saídas claras; return é adequado para resultados reutilizáveis. Type hints comunicam intenções mas não fazem checagem automática em execução. Docstrings descrevem comportamento e exceções.

Exemplo executável

```
def media(notas: list[float]) -> float:
    """Calcula a media de notas nao vazias."""
    if not notas:
        raise ValueError("Sem notas")
    return sum(notas) / len(notas)

assert media([6, 8]) == 7
```

Atenção técnica. Evite misturar input e print com a lógica; assim a função pode ser testada em notebooks ou outros programas.

Exercício de consolidação. Escreva docstring com parâmetros, retorno, limites e exceções.

03 Parâmetros e argumentos mutáveis

Parâmetros posicionais, nomeados, *args e **kwargs permitem APIs flexíveis. Nunca use lista ou dicionário mutável como argumento padrão quando a intenção é reiniciar o valor a cada chamada.

Exemplo executável

```
def adicionar(item, itens=None):
    if itens is None:
        itens = []
    itens.append(item)
    return itens

assert adicionar("A") == ["A"]
assert adicionar("B") == ["B"]
```

Atenção técnica. Os defaults de funções são avaliados uma vez na definição, não a cada chamada.

Exercício de consolidação. Construa um contraexemplo que revele a retenção de estado com itens=[].

04 Escopo, pureza e testes

Funções puras simplificam raciocínio e reprodução. Variáveis globais mutáveis podem criar dependências invisíveis entre células. Teste casos usuais, extremos e inválidos com assert ou unittest.

Exemplo executável

```
def normalizar_valores(valores):
    if not valores:
        return []
    m = max(valores)
    return [v / m for v in valores] if m else [0] * len(valores)

assert normalizar_valores([0, 2, 4]) == [0, .5, 1]
```

Atenção técnica. Resultados reproduzíveis dependem da ordem de execução; reinicie o kernel e execute tudo para validar notebooks.

Exercício de consolidação. Escreva testes de valores vazios, zeros e números negativos.

05 Parâmetros e assinaturas

Uma assinatura bem desenhada especifica como a função deve ser chamada. Parâmetros somente posicionais (/) e somente nomeados (*) ajudam a estabilizar APIs. *args reúne valores posicionais; **kwargs reúne pares nomeados.

Exemplo executável

```
def intervalo(inicio, /, fim, *, passo=1):
    return list(range(inicio, fim, passo))
assert intervalo(0, 5, passo=2) == [0,2,4]
```

Atenção técnica. Flexibilidade excessiva torna APIs difíceis de descobrir e validar; prefira parâmetros explícitos.

Exercício de consolidação. Escreva uma assinatura na qual unidade seja obrigatoriamente nomeada.

06 Funções de ordem superior

Funções são objetos: podem ser passadas como argumentos, retornadas e armazenadas em coleções. map e filter expressam transformações, mas compreensões costumam ser mais legíveis para casos simples. Use callbacks quando a operação deve ser parametrizada.

Exemplo executável

```
def transformar(valores, operacao):
    return [operacao(v) for v in valores]
assert transformar([1,2,3], lambda x: x**2) == [1,4,9]
```

Atenção técnica. Lambdas extensas prejudicam a leitura; use def para funções com várias condições ou regras.

Exercício de consolidação. Escreva transformação que normalize textos por uma função recebida.

07 Erros, logs e contratos

Erros são parte da interface da função. Um ValueError indica valor inadequado; TypeError indica tipo incompatível. Capture exceções quando houver ação de recuperação real; registre contexto em logging sem expor dados sensíveis.

Exemplo executável

```
def proporcao(parte, total):
    if total <= 0:
        raise ValueError("Total deve ser positivo")
    if parte < 0 or parte > total:
        raise ValueError("Parte fora do intervalo")
    return parte / total
assert proporcao(2, 4) == 0.5
```

Atenção técnica. Não substitua exceções por retornos ambíguos como 0 ou None sem definir contrato.

Exercício de consolidação. Defina tipos de exceção para arquivo ausente e dado inválido.

08 Testes unitários e parametrização

Testes devem verificar comportamento observável, não detalhes internos. unittest é biblioteca padrão; testes parametrizados podem ser implementados por subTest. Pequenas funções testáveis se integram melhor a projetos e notebooks.

Exemplo executável

```
import unittest
class TesteMedia(unittest.TestCase):
    def test_media(self):
        for notas, esperado in [([2,4],3),([10],10)]:
            with self.subTest(notas=notas):
                self.assertEqual(sum(notas)/len(notas), esperado)
```

Atenção técnica. Testes que dependem do estado de células anteriores não garantem reprodutibilidade.

Exercício de consolidação. Crie testes que capturem ValueError e verifiquem sua mensagem.

Checklist de revisão

Verifique	Evidência
Conceito	Consegue explicar sem copiar o código?
Execução	O exemplo produz o resultado esperado?
Casos-limite	Entradas inválidas e extremas foram testadas?
Reprodutibilidade	O kernel e as dependências estão registrados?

Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.